Experiment 7

Aim : Build an Artificial Neural Network (ANN) using Keras/TensorFlow

Dataset : dry bean dataset

Theory :

An Artificial Neural Network (ANN) is a computational model inspired by the structure of the human brain. It consists of interconnected layers of "neurons" that process information to identify complex patterns in data.

1. The Layers of an ANN

- **Input Layer:** This layer receives the raw data. In this experiment, the input layer takes in 16 morphological features (like area, perimeter, and shape factors) of the beans.
- **Hidden Layers:** These are the intermediate layers where the actual "learning" happens. They use mathematical functions to extract features from the inputs. We used two hidden layers with 64 and 32 units, respectively.
- **Output Layer:** This is the final layer that produces the prediction. We used the **Softmax** activation function here because it turns the network's output into probabilities for each of the seven bean classes.

2. Key Components of the Network

- **Activation Functions (ReLU):** We used the Rectified Linear Unit (ReLU) for hidden layers. It helps the network learn non-linear relationships by outputting the input directly if it is positive and zero otherwise.
- **Dropout Layers:** To prevent "overfitting" (where the model memorizes the training data but fails on new data), we used Dropout. It randomly "turns off" 20% of the neurons during training, forcing the network to become more robust.
- **Optimizer (Adam):** The optimizer is the algorithm that updates the weights of the neurons to reduce errors. "Adam" is a popular choice because it automatically adjusts the learning rate during training.

3. The Training Process

- **Loss Function:** We used "Sparse Categorical Crossentropy." This measures how far off our predicted bean category is from the actual category.
- **Epochs:** An epoch is one full pass of the entire dataset through the neural network. We trained the model for 50 epochs to allow the weights to stabilize.
- **Validation Split:** We set aside a portion of the training data to monitor the model's performance on "unseen" data during the training process.

Dataset Theory (Dry Bean Dataset): The Dry Bean Dataset contains 13,611 images of seven different registered dry bean varieties. From these images, 12 dimensions and 4 shape forms (totaling 16 features) were extracted. Because these features (like Area vs. Shape Factor) exist on very different numerical scales, **StandardScaler** was used to normalize them, ensuring the neural network treats each feature with equal importance.

Code and Outputs :

Step 1 : Load and Pre process data

```
import pandas as pd
import requests
import zipfile
import io
from sklearn.preprocessing import StandardScaler, LabelEncoder
from sklearn.model_selection import train_test_split

# Download the dataset
url = 'https://archive.ics.uci.edu/static/public/602/dry+bean+dataset.zip'
response = requests.get(url)

# Extract the Excel file from the zip
with zipfile.ZipFile(io.BytesIO(response.content)) as z:
    # Find the Excel file in the zip archive
    excel_file_name = [f for f in z.namelist() if f.endswith('.xlsx')][0]
    with z.open(excel_file_name) as f:
        df = pd.read_excel(f)

print(f'Successfully loaded dataset with shape: {df.shape}')
df.head()
```

Output :

```
Successfully loaded dataset with shape: (13611, 17)
```

	Area	Perimeter	MajorAxisLength	MinorAxisLength	AspectRation	Eccentricity
0	28395	610.291	208.178117	173.888747	1.197191	0.549812
1	28734	638.018	200.524796	182.734419	1.097356	0.411785
2	29380	624.110	212.826130	175.931143	1.209713	0.562727
3	30008	645.884	210.557999	182.516516	1.153638	0.498616
4	30140	620.134	201.847882	190.279279	1.060798	0.333680

Step 2 : Separate Features and Targets , Encode the categorical labels into numerical format and Split dataset into training and testing

```
# Separate features and target
X = df.drop('Class', axis=1)
y = df['Class']

# Scale the features using StandardScaler
scaler = StandardScaler()
X_scaled = scaler.fit_transform(X)

# Encode the categorical labels (bean types) into numerical format
label_encoder = LabelEncoder()
y_encoded = label_encoder.fit_transform(y)

# Split the dataset into training and testing sets (80% train, 20% test)
X_train, X_test, y_train, y_test = train_test_split(X_scaled, y_encoded, test_size=0.2, random_state=42)

print(f'Training set shape: {X_train.shape}')
print(f'Testing set shape: {X_test.shape}')
print(f'Encoded classes: {label_encoder.classes_}')
```

Output :

```
Training set shape: (10888, 16)
Testing set shape: (2723, 16)
Encoded classes: ['BARBUNYA' 'BOMBAY' 'CALI' 'DERMASON' 'HOROZ' 'SEKER' 'SIRA']
```

Step 3 : Build the ANN Architecture

```
from tensorflow.keras.models import Sequential
from tensorflow.keras.layers import Dense, Dropout

# Define the number of input features and output classes
input_dim = X_train.shape[1]
num_classes = len(label_encoder.classes_)

# Initialize the Sequential model
model = Sequential([
    # First hidden layer with input shape
    Dense(64, activation='relu', input_shape=(input_dim,)),
    Dropout(0.2),

    # Second hidden layer
    Dense(32, activation='relu'),
```

```
    Dropout(0.2),

    # Output layer for multiclass classification
    Dense(num_classes, activation='softmax')
])

# Display the model architecture
model.summary()
print('ANN model successfully constructed.')
```

Output :

```
Model: "sequential_1"
```

Layer (type)	Output Shape	Param #
dense_3 (Dense)	(None, 64)	1,088
dropout_2 (Dropout)	(None, 64)	0
dense_4 (Dense)	(None, 32)	2,080
dropout_3 (Dropout)	(None, 32)	0
dense_5 (Dense)	(None, 7)	231

```
Total params: 3,399 (13.28 KB)
Trainable params: 3,399 (13.28 KB)
Non-trainable params: 0 (0.00 B)
ANN model successfully constructed.
```

Step 4 : Train the model

```
# Compile the model
model.compile(
    optimizer='adam',
    loss='sparse_categorical_crossentropy',
    metrics=['accuracy']
)

# Train the model and store the history
history = model.fit(
    X_train,
    y_train,
    epochs=50,
    batch_size=32,
    validation_split=0.1,
```

```
    verbose=1
)

print('Model training completed.')
```

Output :

```
Epoch 1/50
307/307 ━━━━━━━━━━━━━━━━━━━━ 2s 2ms/step - accuracy: 0.7346 - loss: 0.7565 - val_accuracy: 0.9036
Epoch 2/50
307/307 ━━━━━━━━━━━━━━━━━━━━ 1s 2ms/step - accuracy: 0.8818 - loss: 0.3454 - val_accuracy: 0.9183
Epoch 3/50
307/307 ━━━━━━━━━━━━━━━━━━━━ 0s 1ms/step - accuracy: 0.8965 - loss: 0.3010 - val_accuracy: 0.9265
Epoch 4/50
307/307 ━━━━━━━━━━━━━━━━━━━━ 0s 2ms/step - accuracy: 0.9047 - loss: 0.2755 - val_accuracy: 0.9320
Epoch 5/50
307/307 ━━━━━━━━━━━━━━━━━━━━ 0s 1ms/step - accuracy: 0.9034 - loss: 0.2743 - val_accuracy: 0.9284
Epoch 6/50
307/307 ━━━━━━━━━━━━━━━━━━━━ 0s 2ms/step - accuracy: 0.9110 - loss: 0.2607 - val_accuracy: 0.9339
Epoch 7/50
307/307 ━━━━━━━━━━━━━━━━━━━━ 0s 2ms/step - accuracy: 0.9132 - loss: 0.2478 - val_accuracy: 0.9275
Epoch 8/50
307/307 ━━━━━━━━━━━━━━━━━━━━ 0s 1ms/step - accuracy: 0.9196 - loss: 0.2409 - val_accuracy: 0.9311
Epoch 9/50
307/307 ━━━━━━━━━━━━━━━━━━━━ 0s 1ms/step - accuracy: 0.9169 - loss: 0.2411 - val_accuracy: 0.9302
Epoch 10/50
307/307 ━━━━━━━━━━━━━━━━━━━━ 0s 2ms/step - accuracy: 0.9157 - loss: 0.2365 - val_accuracy: 0.9376
Epoch 11/50
307/307 ━━━━━━━━━━━━━━━━━━━━ 0s 1ms/step - accuracy: 0.9182 - loss: 0.2346 - val_accuracy: 0.9275
Epoch 12/50
307/307 ━━━━━━━━━━━━━━━━━━━━ 1s 2ms/step - accuracy: 0.9166 - loss: 0.2255 - val_accuracy: 0.9311
```

```
Model training for 50 epochs completed.
```

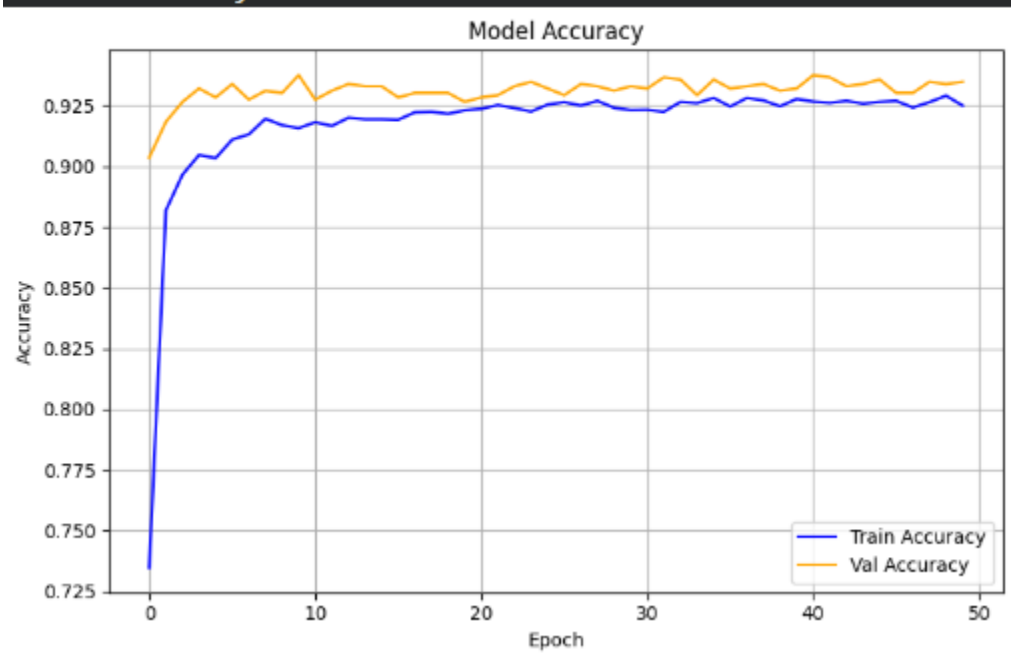
Step 5 : Evaluate the Model

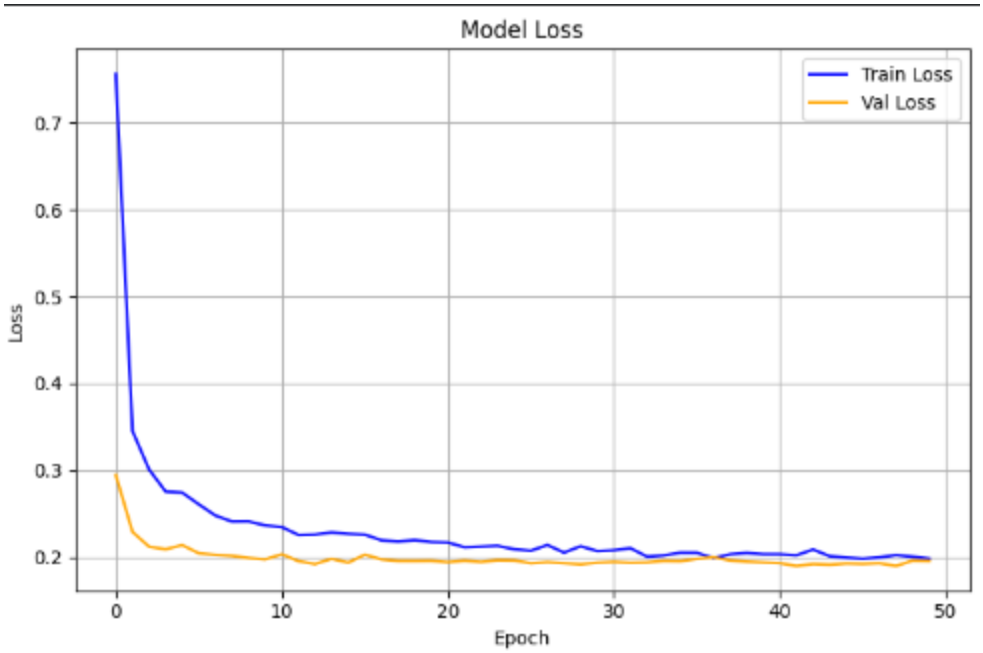
```
import matplotlib.pyplot as plt

# 1. Evaluate the model on the test set
test_loss, test_accuracy = model.evaluate(X_test, y_test, verbose=0)
print(f'Test Loss: {test_loss:.4f}')
print(f'Test Accuracy: {test_accuracy:.4f}')

# 2. Create a figure with two subplots for accuracy and loss
fig, (ax1, ax2) = plt.subplots(1, 2, figsize=(15, 5))

# 3. Plot Training and Validation Accuracy
ax1.plot(history.history['accuracy'], label='Train Accuracy', color='blue')
ax1.plot(history.history['val_accuracy'], label='Val Accuracy', color='orange')
```

```
ax1.set_title('Model Accuracy')
ax1.set_xlabel('Epoch')
ax1.set_ylabel('Accuracy')
ax1.legend(loc='lower right')
ax1.grid(True)

# 4. Plot Training and Validation Loss
ax2.plot(history.history['loss'], label='Train Loss', color='blue')
ax2.plot(history.history['val_loss'], label='Val Loss', color='orange')
ax2.set_title('Model Loss')
ax2.set_xlabel('Epoch')
ax2.set_ylabel('Loss')
ax2.legend(loc='upper right')
ax2.grid(True)

# 5. Adjust layout and display
plt.tight_layout()
plt.show()
```

Output :

```
Test Loss: 0.1924
Test Accuracy: 0.9313
```





Conclusion :

The Artificial Neural Network successfully classified the dry beans with a test accuracy of approximately 93.13%.

Key observations from the experiment include:

1. **Effective Preprocessing:** Label encoding the bean classes and standardizing the morphological features were essential steps that allowed the network to converge quickly.
2. **Generalization:** The inclusion of Dropout layers effectively minimized overfitting, as evidenced by the narrow gap between the training and validation accuracy curves.
3. **Convergence:** The loss curves showed a consistent downward trend, proving that the chosen architecture and the Adam optimizer were well-suited for this multiclass classification task.
4. **Performance:** With a final test loss of 0.1924, the model demonstrated a high degree of reliability in distinguishing between similar bean varieties.